ISM2411 — Lab Week 03

Product Pricer with f-strings — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 03 · Unit 1 · Foundations

Contents

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 03 Lab — Product Pricer with f-strings
Unit	Unit 1 · Foundations
Class length	75 minutes
Format	Live code-along
Prerequisites	Module 02: terminal navigation, running a `.py` file, reading a basic error message
Student-facing lab page	markumreed.github.io/ism2411 — week03_lab
Exercises covered	Exercises 1–5 (required) + Stretch A/B (as time allows)
Submission	`pricer.py` to Canvas, exercises 1–4 as one runnable script, comments separating each exercise

Module 02 got students *running* a script someone else wrote (essentially — `hello.py` is one line). This is the first lab where students write meaningfully structured code themselves: multiple variables of different types, working together, formatted into professional-looking output. The two ideas to protect time for are (1) that Python variables carry a *type*, and types determine behavior (+ means something completely different for strings than for numbers), and (2) the f-string format-spec mini-language, which will be reused in nearly every remaining module this semester.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Create variables of `str`, `float`, `int`, and `bool` type and confirm a variable's type with `type()`.
2. Build an f-string that both computes a value inline (`{unit_price * quantity}`) and formats it for display (`:.2f`).
3. Convert `input()`'s string return value to the correct numeric type before doing math with it.
4. Explain why `"5" + "3"` and `5 + 3` produce different results, and connect that directly to why `input()` conversion is mandatory, not optional.
5. Use the Python interactive interpreter (the REPL) as a quick scratch space for testing small expressions, separate from running a saved `.py` file.

3 Before Class — Setup Checklist

- ☐ Open a terminal and an editor (VS Code) you can screen-share, plus have the Python interactive interpreter (`python3` with no filename) ready to demo for Exercise 5.

- ☐ Pre-create an empty `pricer.py` — everything is typed live.
- ☐ Confirm your own Python version is 3.10+ before class (f-string `.1%`/`,` format specs and `type()` output format are stable across recent versions, but worth a sanity check).
- ☐ Decide your own product/price/cost numbers ahead of time so your live output is clean and round enough to read from the back row (this guide uses Notebook / $4.99 / $3.29 / 12 units throughout — feel free to substitute your own, all output below is verified against these exact numbers).

4 Materials Needed

- Instructor laptop + terminal + editor, Python 3.10+
- Students: `pricer.py` open in the same project folder structure from Modules 01–02
- No external packages needed for the required exercises; Stretch content in Module 02 already introduced `pip install`, not needed again here

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:05	Welcome, recap Module 02, frame today's file	5
0:05–0:13	Exercise 1 — Variables and types	8
0:13–0:21	Exercise 2 — f-string practice	8
0:21–0:31	Exercise 3 — Input version	10
0:31–0:43	Exercise 4 — Multi-variable product card	12
0:43–0:55	Exercise 5 — Type investigation (REPL)	12
0:55–1:10	Stretch A — Three-product comparison	15
1:10–1:15	Stretch B preview + wrap-up, reflection, submission checklist	5

Five required exercises plus Stretch A comfortably fill 75 minutes at a code-along pace; Stretch B (format-spec exploration) is a fast, discussion-style closer rather than a full 15-minute block — if the room is behind schedule, cut Stretch B to a single one-sentence teaser rather than skipping Exercise 5 or Stretch A.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:05)

Say to the class:

> *"Module 02 got you running a one-line script someone basically handed you. Today you write a real script from scratch: four variables of different types, working together, printed as something that looks like it belongs in a business report. The single most important idea today is that every value in Python has a* type*, and the type changes what an operator like* `+` *actually does. Get that wrong and your program doesn't crash — it just does something you didn't expect. Sound familiar? That's the same 'silent wrong answer' theme from every module this semester."*

Do: Open `pricer.py`, type the header comment:

```
# pricer.py
# ISM2411 Module 03 Lab — Product Pricer with f-strings
```

6.2 Exercise 1 — Variables and Types (0:05–0:13, 8 min)

Teaching goal: Every value has a type; `type()` is how you ask Python what it thinks a value is, and the four types here (`str`, `float`, `int`, `bool`) are the four students will use constantly all semester.

Say to the class:

> *"Four variables, four different types. We're going to ask Python directly what type each one is, rather than just trusting our own guess."*

Live-code this:

```
# --- Exercise 1 ---
product = "Notebook"
unit_price = 4.99
quantity = 12
in_stock = True
print(type(product))
print(type(unit_price))
print(type(quantity))
print(type(in_stock))
```

Line-by-line explanation:

- `product = "Notebook"` — a **string** (`str`): text, always wrapped in quotes. Point out: the

quotes are what make it a string — `product = Notebook` (no quotes) would be a `NameError`, since Python would try to interpret `Notebook` as a variable name that doesn't exist. Worth running that broken version live for 10 seconds; it's a very common typo.
- `unit_price = 4.99` — a **float**: any number with a decimal point, even if that decimal is `.00`. `4.99` is a float regardless of the value.
- `quantity = 12` — an **int**: a whole number, no decimal point at all. `12` is an `int`; `12.0` would be a `float` — the presence of the decimal point is the only thing that matters to Python here, not whether the value happens to be a whole number.
- `in_stock = True` — a **bool**: exactly `True` or `False`, capitalized, no quotes (a callback to Module 03's — sorry, actually this is the *first* time booleans appear this semester as literal values students type themselves, so treat it as new: this is not the string `"True"`, it is the keyword `True`).
- `print(type(product))` — `type()` is a built-in function that takes any value and returns its type as a special kind of value, which prints as `<class 'str'>`. Say explicitly: `<class 'str'>` is Python's *display format* for "this is the string type" — the word `class` is not something to worry about yet (types are implemented as classes under the hood; that's a Module far in the future, if ever, for a business-Python course — don't rabbit-hole here).

Run it. Expected output:

```
<class 'str'>
<class 'float'>
<class 'int'>
<class 'bool'>
```

Common student mistakes to watch for:

- Forgetting quotes around the string value, producing a `NameError: name 'Notebook' is not defined` — a good live demo, as noted above.
- Writing `12.00` for quantity out of habit (since it's dollars-adjacent in their head) and being confused that `type()` reports `float`, not `int` — reinforce: Python doesn't know or care that a quantity "should" be a whole number; it only looks at whether you typed a decimal point.
- Writing `true` (lowercase) instead of `True` — Python is case-sensitive and `true` is not a recognized keyword, so this raises a `NameError` just like the unquoted string above. Worth noting the *same* error type (`NameError`) can come from two totally different mistakes — the error category tells you "undefined name," not *why* it's undefined.

Check for understanding: "If I write `quantity = "12"` instead — with quotes — what does `type(quantity)` report, and what would go wrong later if I tried to do math with it?" (`<class 'str'>` — and this is the exact setup for Exercise 3's `input()` conversion problem, so flag that connection explicitly now.)

6.3 Exercise 2 — f-string Practice (0:13–0:21, 8 min)

Teaching goal: An f-string can *compute* a value inline, not just display an already-computed variable — this is new relative to Module 01's f-string usage (which only displayed pre-computed variables).

Say to the class:

> *"So far every f-string you've written has displayed a variable that was already computed on a previous line. Today's version does the math* inside *the braces, live."*

Live-code this:

```
# --- Exercise 2 ---
product = "Notebook"
unit_price = 4.99
quantity = 12
print(f"{quantity} units of {product} at ${unit_price} each = ${unit_price * quantity:.2f}")
```

Line-by-line explanation:

- The first three lines reuse Exercise 1's variables — some instructors like to point out this is a fresh script run (Exercise 1's `print(type(...))` lines are gone), so nothing carries over between exercises unless it's re-typed; this matters because Exercise 4 will build a *combined* script where these do carry over within the same file.
- `f"{quantity} units of {product} at ${unit_price} each = ${unit_price * quantity:.2f}"` — walk this string left to right, since it has four separate substitutions:
  - `{quantity}` — plain substitution, no format spec, prints `12`.
  - `{product}` — plain substitution, prints `Notebook`.
  - `{unit_price}` — plain substitution, prints `4.99` (no format spec needed here since `4.99` already has exactly two decimals; flag that this is a coincidence of the input value, not something to rely on — if `unit_price` were `5`, this would print `5`, not `5.00`).
  - `{unit_price * quantity:.2f}` — this is the one doing real work: `unit_price * quantity` is evaluated first (`4.99 * 12 = 59.88`), *then* the `:.2f` format spec is applied to that computed result. Emphasize: the calculation and the formatting are two separate steps happening in the same set of braces, in that order.

Run it. Expected output:

```
12 units of Notebook at $4.99 each = $59.88
```

Common student mistakes to watch for:

- Trying to put the format spec on the wrong part of the expression, e.g. `{unit_price:.2f * quantity}` — this is a `SyntaxError` because the format spec has to come *after* the complete expression, not in the middle of it. Show this error live; the traceback is a good one to read

together since format-spec placement mistakes recur all semester.

- Forgetting the `$` is a literal character and trying to make it "part of" the number somehow — reinforce that `$` outside the braces is just text, same lesson as Module 04's revenue formatting (if this course reaches that module later in sequence) or simply state it fresh here.

Check for understanding: "What would this line print if I changed `unit_price` to `5` (no decimal) — walk me through both substitutions that involve it." (`{unit_price}` alone would print `5`; `{unit_price * quantity:.2f}` would still print `60.00`, because the format spec is doing the work of adding the trailing zeros regardless of the input's original precision.)

6.4 Exercise 3 — Input Version (0:21–0:31, 10 min)

Teaching goal: Convert Exercise 2 from hardcoded values to real user input — and hit the type-conversion requirement head-on, since this is the exercise the reflection questions specifically ask students to reason about.

Say to the class:

> *"Same summary line as Exercise 2, but now the values come from whoever runs the script. Watch closely — `input()` is going to hand us exactly the wrong type for two of these three values, and I want you to predict the failure before we hit it."*

Live-code this in two passes. First, deliberately *without* conversion, to let the room see the failure:

```
# --- Exercise 3 (broken first pass — do not leave this in the final file) ---
product = input("Product: ")
unit_price = input("Unit price: ")
quantity = input("Quantity: ")
print(f"{quantity} units of {product} at ${unit_price} each = ${unit_price * quantity:.2f}")
```

Run it with sample input `Notebook`, `4.99`, `12`. **This raises:**

```
TypeError: can't multiply sequence by non-int of type 'str'
```

Explain the error, out loud, slowly: `unit_price * quantity` is trying to multiply two *strings* together (both `input()` results came back as `str`, unconverted). Python does support `*` between a string and an *int* — but that means something completely different from arithmetic multiplication (it repeats the string that many times, e.g. `"ab" * 3` gives `"ababab"`) — and here Python can't even do that, because *both* operands are strings, and string-times-string isn't defined at all. This is worth stating precisely because "can't multiply sequence by non-int" is a genuinely confusing error message to a beginner; unpacking it explicitly is the actual teaching moment.

Now fix it — second pass:

```
# --- Exercise 3 ---
product = input("Product: ")
unit_price = float(input("Unit price: "))
quantity = int(input("Quantity: "))
print(f"{quantity} units of {product} at ${unit_price} each = ${unit_price * quantity:.2f}")
```

Line-by-line explanation of the fix:

- `unit_price = float(input("Unit price: "))` — `input()` returns a string; `float()` wraps that call and converts the result. Same inside-out reading pattern as prior modules.
- `quantity = int(input("Quantity: "))` — same pattern with `int()`, since a quantity should

be a whole number.
- `product` stays unconverted, deliberately — it's meant to remain text.

Run it with the same sample input. Expected output:

```
12 units of Notebook at $4.99 each = $59.88
```

Common student mistakes to watch for:

- Converting `product` with `str()` out of an over-corrected instinct — harmless (it's already a string), but a good moment to say explicitly: conversion is about changing what a value *is*, and converting a string to a string is a no-op, not an error, just unnecessary.
- Converting `quantity` with `float()` instead of `int()` — runs fine, but changes `type(quantity)` to `float`, which will print `12.0` instead of `12` in the summary line unless the format spec compensates. Ask the room to predict what changes in the output if you make this swap, live.

Check for understanding: This is the exact question from tonight's reflection prompt — ask it now, out loud, so students have already rehearsed the answer before they see it on the lab page: "What exactly goes wrong, and what's the *exact error type*, if you forget to convert `unit_price` to `float`?" (A `TypeError`, specifically about not being able to multiply a string by another string — make someone say "TypeError," not just "it breaks.")

6.5 Exercise 4 — Multi-Variable Product Card (0:31–0:43, 12 min)

Teaching goal: Combine everything so far — multiple variables, a derived value (margin), and several format specs — into one clean, multi-line report. This is the exercise that most resembles what "real" business scripts look like.

Say to the class:

> *"One more variable — `unit_cost` — and we're computing a derived value, margin, the same formula from the reading. Five lines of output, cleanly aligned."*

Live-code this:

```
# --- Exercise 4 ---
product = "Notebook"
unit_price = 4.99
unit_cost = 3.29
quantity = 12
revenue = unit_price * quantity
margin = (unit_price - unit_cost) / unit_price

print(f"Product:  {product}")
print(f"Price:    ${unit_price:.2f}")
print(f"Qty:      {quantity}")
print(f"Revenue:  ${revenue:.2f}")
print(f"Margin:   {margin:.1%}")
```

Line-by-line explanation:

- `unit_cost = 3.29` — the new variable this exercise introduces.
- `revenue = unit_price * quantity` — same formula pattern as before, stored in its own variable this time rather than computed inline inside the f-string. Ask the room: "Why compute this on its own line instead of inline like Exercise 2 did?" (Because it's used more than once conceptually — well, not literally reused here, but the style point is that once a calculation has a name of its own, like `revenue`, the code that prints it becomes easier to read. This is a genuine style upgrade worth naming.)
- `margin = (unit_price - unit_cost) / unit_price` — identical formula to prior margin calculations; the parentheses are required for the same order-of-operations reason as always — flag it briefly, don't re-derive it from scratch if this room has seen it before.
- The five `print()` lines — each uses a **label padded with extra spaces** (`"Product:  "`, `"Price:    "`, etc.) so the values line up in a column when printed — point out this is a *manual* alignment technique (counting spaces by eye), and it's fragile: if a product name is much longer, the alignment breaks. This is worth flagging as a real limitation, and a good bridge to a "there are better ways to do this" comment (string `.ljust()`/`.rjust()` or f-string

alignment specs like `{label:<10}` exist, but are out of scope for this lab — mention only if a curious student asks).
- Format specs recap, quickly: `${unit_price:.2f}` for currency, plain `{quantity}` for an integer needing no formatting, `${revenue:.2f}` for currency again, `{margin:.1%}` for the percentage — all four specs have now appeared in this course, and this line is a good moment to say so explicitly.

Run it. Expected output:

```
Product:  Notebook
Price:    $4.99
Qty:      12
Revenue:  $59.88
Margin:   34.1%
```

Common student mistakes to watch for:

- Misaligning the manual spacing (extra or missing spaces in the label strings) — cosmetic, not a functional bug, but worth walking the room for since "does this look like a report" is literally what the exercise is testing.
- Using `unit_price` instead of `unit_cost` in the margin formula by copy-paste habit from earlier exercises — produces `margin = 0.0` (since `unit_price - unit_price` is always zero), a good "silent wrong answer, and a suspiciously round one" example — `0.0%` margin should visibly look wrong to anyone glancing at the output.

Check for understanding: "If `unit_cost` were *higher* than `unit_price` — selling at a loss — what would this script print for margin, and would it crash?" (A negative percentage, e.g. `-10.0%` — no crash, since there's nothing mathematically invalid about a negative margin; this is a good moment to note that Python will happily compute a nonsensical business result without any complaint, which is the whole "silent wrong answer" theme again.)

6.6 Exercise 5 — Type Investigation in the REPL (0:43–0:55, 12 min)

Teaching goal: Introduce the Python interactive interpreter (REPL) as a separate tool from running a saved script — a fast scratchpad for testing small expressions — and use it to directly observe the `str`-vs-`int` addition distinction that motivates every type-conversion rule so far.

Say to the class:

> *"Everything today has been inside a saved file you run with `python3 pricer.py`. There's a second way to run Python: type `python3` with no filename, and you get an interactive prompt where you can type one expression at a time and see the result immediately. This is where you'll quickly test an idea before committing it to a script."*

Do, live, in the terminal:

```
$ python3
>>> type("12")
<class 'str'>
>>> type(12)
<class 'int'>
>>> type(12.0)
<class 'float'>
>>> type(True)
<class 'bool'>
>>> "5" + "3"
'53'
>>> 5 + 3
8
```

Explain each result as it appears:

- `type("12")` → `<class 'str'>` — the quotes make `"12"` a string that happens to *look like* a number, but Python treats it purely as text.
- `type(12)` → `<class 'int'>`, `type(12.0)` → `<class 'float'>` — reinforce the decimal-point rule from Exercise 1 one more time, now via the REPL instead of a script.
- `type(True)` → `<class 'bool'>` — consistent with Exercise 1.
- `"5" + "3"` → `'53'` — **this is the payoff of the whole exercise.** `+` between two strings means **concatenation** (gluing text together), not addition. `"5"` and `"3"` are text, not numbers, so `+` sticks them end to end into the four-character string `'53'` (note: displayed with single quotes in the REPL — that's just how the REPL shows you it's a string, distinct from `print()`, which would show `53` with no quotes; worth a 10-second aside if a sharp student asks why the quotes appear here but didn't in earlier `print()` output).
- `5 + 3` → `8` — the same `+` symbol, but now both operands are `int`, so it means ordinary arithmetic addition.

State the connective idea explicitly, since this is the exercise's whole point:

> *"Same symbol, `+`, two completely different behaviors depending on type. This is* exactly *why `input()` results have to be converted before you do math with them — `input()` always returns a string, so `price + tax` where both came straight from `input()` wouldn't add two numbers, it would glue two pieces of text together. And it wouldn't necessarily crash — if both look like numbers, you'd get a weird-looking-but-valid string and might not notice anything was wrong until much later."*

Common student mistakes to watch for:

- Typing `exit()` or closing the terminal window entirely instead of using `Ctrl+D` (Mac/Linux) or `Ctrl+Z` then Enter (Windows) to leave the REPL — not wrong, just slower; mention the shortcut once.
- Confusion about why the REPL echoes back a *value* (like `'53'`) for a bare expression but does nothing for a bare `print(...)` beyond the printed text itself — the REPL auto-displays the result of the last expression typed, which is a REPL-only convenience; a saved script never does this automatically, which is precisely why `print()` is required in every script exercise all semester.

Check for understanding: "Predict, before I run it: what does `"5" + 3` — a string plus an int, not two strings — do?" (It raises a `TypeError: can only concatenate str (not "int") to str` — demonstrate this live if time allows; it's a natural bridge into "Python refuses to guess which behavior you meant when the types don't match cleanly," which is a genuinely reassuring fact for beginners worried about silent bad behavior everywhere.)

6.7 Stretch A — Three-Product Comparison (0:55–1:10, 15 min)

Teaching goal: Scale Exercise 4's single product card to three products using parallel variable naming, and use comparison operators to pick a winner — direct rehearsal for the more general "many records" processing that loops (a later module) will handle more elegantly.

Say to the class:

> *"Three products, three full sets of variables, three margins — and a final line declaring the winner. This previews a real pain point: right now, with no loops yet, three products means typing almost the same code three times. Notice that discomfort — it's setting up why we'll want a better tool for this soon."*

Live-code this:

```
# --- Stretch A ---
product1, price1, cost1, qty1 = "Notebook", 4.99, 3.29, 12
product2, price2, cost2, qty2 = "Pen", 1.99, 0.85, 50
product3, price3, cost3, qty3 = "Stapler", 12.49, 7.10, 5

margin1 = (price1 - cost1) / price1
margin2 = (price2 - cost2) / price2
margin3 = (price3 - cost3) / price3

for name, price, qty, margin in [
    (product1, price1, qty1, margin1),
    (product2, price2, qty2, margin2),
    (product3, price3, qty3, margin3),
]:
    print(f"Product: {name} | Price: ${price:.2f} | Qty: {qty} | "
          f"Revenue: ${price * qty:.2f} | Margin: {margin:.1%}")

best_margin, best_product = max([(margin1, product1), (margin2, product2), (margin3, product
print(f"Highest margin: {best_product} at {best_margin:.1%}")
```

Line-by-line explanation:

- The three `productN, priceN, costN, qtyN = ...` lines — tuple unpacking again (Exercise 3's product card, scaled to three parallel sets). Point out explicitly: **numbered variable names like this are a known anti-pattern** — the exercise is teaching students to feel that pain, not endorsing it as good style. If a student asks "isn't there a better way," the honest answer is "yes, a list of dictionaries or a loop-friendly structure — that's coming in a later module; today's exercise is intentionally the 'hard way' so the better way lands harder when you see it."
- The `for` loop over a list of tuples is one legitimate way to avoid writing three nearly-identical

`print()` calls — if your section hasn't covered `for` loops yet, **replace this with three explicit, separately-typed `print()` calls instead** (one per product, copy-pasted with the numbers changed) so the loop syntax doesn't introduce an unannounced concept; the accompanying appendix answer key shows both versions.
- `max([(margin1, product1), (margin2, product2), (margin3, product3)])` — `max()` on a list of tuples compares tuples element by element, so it compares the *margins first* (since margin is listed first in each tuple) and only looks at the product name to break ties. This is a genuinely useful trick worth naming explicitly, since it's not obvious the first time you see it: `max()` doesn't need you to write your own comparison logic here, because tuple comparison does it for you.

Run it. Expected output:

```
Product: Notebook | Price: $4.99 | Qty: 12 | Revenue: $59.88 | Margin: 34.1%
Product: Pen | Price: $1.99 | Qty: 50 | Revenue: $99.50 | Margin: 57.3%
Product: Stapler | Price: $12.49 | Qty: 5 | Revenue: $62.45 | Margin: 43.2%
Highest margin: Pen at 57.3%
```

If `for` loops are not yet in scope for your section, use this simpler, fully explicit version instead (same numbers, same output for the first three lines):

```
print(f"Product: {product1} | Price: ${price1:.2f} | Qty: {qty1} | "
      f"Revenue: ${price1 * qty1:.2f} | Margin: {margin1:.1%}")
print(f"Product: {product2} | Price: ${price2:.2f} | Qty: {qty2} | "
      f"Revenue: ${price2 * qty2:.2f} | Margin: {margin2:.1%}")
print(f"Product: {product3} | Price: ${price3:.2f} | Qty: {qty3} | "
      f"Revenue: ${price3 * qty3:.2f} | Margin: {margin3:.1%}")
```

Have the class determine the winner "by hand" (comparing the three printed margins visually) rather than computing it in code, if you're avoiding `max()` on tuples as too advanced for today.

Common student mistakes to watch for:

- Mixing up which numbered variable belongs to which product mid-script (e.g., using `cost2` in a formula meant for product 3) — a classic consequence of the numbered-variable anti-pattern; when you see it, this is the moment to really land the "wouldn't this be so much easier and safer with a list or a loop variable" point.

Check for understanding: "If I added a fourth product, what has to change about this script?" (Every numbered-variable line needs a new set — `product4, price4, cost4, qty4` — and the loop's list or the three manual print calls need a fourth entry too. Get someone to say out loud that this doesn't scale — that's the intended discomfort.)

6.8 Stretch B Preview — Format Spec Exploration (as time allows)

Frame it as a quick, verbal walkthrough rather than a full live-coded block if time is short — this is designed as REPL exploration, not a script:

```
>>> f"{1234567:.2f}"
'1234567.00'
>>> f"{1234567:,.0f}"
'1,234,567'
>>> f"{0.3456:.1%}"
'34.6%'
>>> f"{42:05d}"
'00042'
>>> f"{'hello':>20}"
'               hello'
```

One sentence each, said out loud: `.2f` forces two decimals even on a whole number; `,.0f` adds thousands separators with zero decimals; `.1%` multiplies by 100 and appends a percent sign, one decimal digit; `05d` pads an integer with leading zeros to a total width of 5 characters (useful for things like order numbers or zip codes); `>20` right-aligns text within a 20-character-wide field (useful for the "columns that line up" problem Exercise 4's manual spacing was working around by hand). If time is genuinely tight, this closing line alone is worth saying: "Everything in that `:` after the value in an f-string is its own small formatting language — you've now seen currency, percentages, thousands separators, zero-padding, and alignment. That's most of what you'll need all semester."

7 Wrap-Up (last ~5 minutes)

Review the reflection questions out loud:

1. *Forgetting to convert price from string to float in Exercise 3* — the exact error is `TypeError: can't multiply sequence by non-int of type 'str'`, and the answer should connect it to the `unit_price * quantity` line specifically, not just say "it errors."
2. *Why `"5" + "3"` gives `"53"`, and how this explains `input()` conversion* — a strong answer states explicitly that `+` behaves differently based on operand type, and that `input()` always returns strings regardless of what the user typed.
3. *A real career scenario for variables/types/f-strings* — no wrong answer; push for a scenario specific enough to name what the variables would be (not just "I'd use it for my job").

Review the submission checklist together:

- ☐ File is named exactly `pricer.py`
- ☐ Contains Exercises 1–4 as one runnable script, in order
- ☐ Each exercise has a comment separating it from the next
- ☐ All numeric output uses an appropriate f-string format spec
- ☐ Script runs top to bottom with no errors

Preview Module 04: "Today's `unit_price * quantity` and margin formulas come back next module, at real business scale — revenue, discounts, and the operators that combine into full financial logic. You'll also meet the *other* division operator, `//`, and see exactly how a missing parenthesis can silently break a formula like today's margin calculation."

8 Appendix A — Full Answer Key (`pricer.py`)

```
# pricer.py
# ISM2411 Module 03 Lab — Product Pricer with f-strings

# --- Exercise 1 ---
product = "Notebook"
unit_price = 4.99
quantity = 12
in_stock = True
print(type(product))
print(type(unit_price))
print(type(quantity))
print(type(in_stock))

# --- Exercise 2 ---
```

```python
product = "Notebook"
unit_price = 4.99
quantity = 12
print(f"{quantity} units of {product} at ${unit_price} each = ${unit_price * quantity:.2f}")

# --- Exercise 3 ---
product = input("Product: ")
unit_price = float(input("Unit price: "))
quantity = int(input("Quantity: "))
print(f"{quantity} units of {product} at ${unit_price} each = ${unit_price * quantity:.2f}")

# --- Exercise 4 ---
product = "Notebook"
unit_price = 4.99
unit_cost = 3.29
quantity = 12
revenue = unit_price * quantity
margin = (unit_price - unit_cost) / unit_price

print(f"Product:  {product}")
print(f"Price:    ${unit_price:.2f}")
print(f"Qty:      {quantity}")
print(f"Revenue:  ${revenue:.2f}")
print(f"Margin:   {margin:.1%}")
```

Exercise 5 (REPL session, not part of `pricer.py`):

```
>>> type("12")
<class 'str'>
>>> type(12)
<class 'int'>
>>> type(12.0)
<class 'float'>
>>> type(True)
<class 'bool'>
>>> "5" + "3"
'53'
>>> 5 + 3
8
```

9 Appendix B — Extra Practice (only if the class finishes early)

Five required exercises plus Stretch A already fill the full 75 minutes at a normal teaching pace. If a section moves unusually fast, use this instead of pulling in Stretch B early:

Extra — a second product card, different numbers. `product = "Desk Lamp"`, `unit_price = 24.50`, `unit_cost = 15.80`, `quantity = 8`. Have students build the same five-line card independently. (Revenue: `$196.00`. Margin: `35.5%`.)

Extra — a second REPL round, different values. Have students predict, then verify in the REPL: `type(3.0)`, `"7" + "7"` vs `7 + 7`, and `"3" * 4` (a string times an int — this one is new: it repeats the string, giving `'3333'`, not `12`). This last one is a genuinely good closer, since it shows `*` has its *own* dual-behavior story, parallel to `+`'s, and previews that "the operator's meaning depends on the operand types" is a general Python rule, not a one-off quirk of `+`.